

MARWADI UNIVERSITY

Rajkot-Morbi Road, At & Po. Gauridad, Rajkot-360003, Gujarat, India

PROJECT REPORT

ML Model Workbench — A Multi-Algorithm Regression Comparison GUI

By

Viraj Vaghasiya (92301733027)

Bhavin Muchhala (92301733043)

Under the guidance of

Prof. Nishith Kotak

A Project Submitted to

Marwadi University in Partial Fulfillment of the Requirements for the subject

Artificial Intelligence (01CT0616)

May 2026



Marwadi
University
Marwadi Chandarana Group



MARWADI UNIVERSITY

Rajkot-Morbi Road, At & Po. Gauridad,
Rajkot-360003, Gujarat, India

Abstract

This project presents the ML Model Workbench, a desktop-based graphical user interface (GUI) built using Python and Tkinter that enables users to train, compare, and evaluate multiple machine learning regression models on any CSV dataset. The system integrates eleven regression algorithms — including Linear Regression, Decision Tree, Random Forest, Bagging, Gradient Boosting, KNN, SVR (RBF), Extra Trees, AdaBoost, LightGBM, and XGBoost — wrapped in standardized scikit-learn pipelines with automatic StandardScaler preprocessing.

The application provides an intuitive workflow from data loading to model evaluation, with interactive visualizations powered by Matplotlib. Key features include real-time model training progress, comparative metrics tables (R^2 , RMSE, MSE), ratio analysis (train/test splits), prediction plots, residual analysis, 3D comparison views, and export functionality. The system is designed to make machine learning accessible to both beginners and practitioners by providing clear visual feedback and a quick-start guide.

Results demonstrate that ensemble methods (Extra Trees, XGBoost, Random Forest) consistently achieve superior R^2 scores compared to linear baselines, while the interactive comparison framework enables rapid model selection tailored to the dataset. This project aligns with the subject Artificial Intelligence (01CT0616) and applies real-world ML techniques including feature engineering, model selection, hyperparameter tuning, and performance benchmarking.

Table of Contents

Abstract	2
Table of Contents	3
List of Figures and Tables	5
Figures.....	5
Tables	5
1. Introduction	6
1.1 Problem Statement	6
1.2 Relevance to Artificial Intelligence	6
1.3 Objectives.....	6
1.4 Scope of the Project.....	6
2. Methodology	8
2.1 AI Techniques Used	8
2.2 Model Selection.....	8
2.3 Algorithms and Mathematical Background.....	8
2.3.1 Linear Regression.....	8
2.3.2 Decision Tree Regression.....	9
2.3.3 Random Forest / Extra Trees / Bagging	9
2.3.4 Gradient Boosting (GBM, XGBoost, LightGBM, AdaBoost).....	9
2.3.5 Support Vector Regression (SVR).....	9
2.3.6 K-Nearest Neighbours Regression.....	9
2.4 Dataset Details.....	9
2.5 Tools and Libraries Used.....	10
3. System Design and Architecture	11
3.1 Block Diagram / Flowchart	11
3.2 Data Pipeline	11
3.3 Modules and Components	11
4. Implementation.....	13
4.1 Code Snippets.....	13
4.1.1 Model Builder with Graceful Fallback (XGBoost).....	13
4.1.2 Training Pipeline with Metrics	13
4.1.3 Ratio Analysis	13
4.2 Model Training Process.....	14
4.3 Evaluation Metrics	14
5. Discussion	15
5.1 Summary of Work.....	15
5.2 Challenges Faced.....	15
5.3 Success Criteria Met.....	15
5.4 Limitations	15

6. Future Work	17
6.1 Scope for Improvements.....	17
6.2 Possible Real-World Deployment Strategies.....	17
Appendix	18
A. Full Code.....	18
B. Project Links.....	18
C. Additional Information	18
C.1 Installation Requirements	18
C.2 Supported Operating Systems.....	18
C.3 Known Fallback Behaviour	19

List of Figures and Tables

Figures

- Figure 1 — System Architecture Block Diagram
- Figure 2 — Data Pipeline Flowchart
- Figure 3 — Model Training and Evaluation Workflow
- Figure 4 — GUI Main Window Layout
- Figure 5 — Model Comparison Bar Chart (R^2 Scores)
- Figure 6 — Prediction vs Actual Plot
- Figure 7 — Residual Distribution Plot
- Figure 8 — Ratio Analysis Chart
- Figure 9 — 3D Model Comparison View

Tables

- Table 1 — Supported Regression Models and Parameters
- Table 2 — Dataset Summary Statistics
- Table 3 — Model Performance Metrics (R^2 , RMSE, MSE)
- Table 4 — Train/Test Ratio Analysis Results
- Table 5 — Tools and Libraries Summary
- Table 6 — Evaluation Criteria

1. Introduction

1.1 Problem Statement

In the domain of supervised machine learning, practitioners routinely face the challenge of selecting the most appropriate regression model for a given dataset. This selection process typically requires writing repetitive boilerplate code, managing multiple library dependencies, and manually comparing model outputs — a process that is time-consuming, error-prone, and inaccessible to non-programmers.

There is no unified, user-friendly desktop tool that allows users to load arbitrary tabular datasets, train multiple regression algorithms simultaneously, and visually compare their performance without writing code. This gap is especially acute in academic and research settings where instructors and students need rapid prototyping capabilities.

1.2 Relevance to Artificial Intelligence

This project is directly relevant to Artificial Intelligence as it implements, compares, and evaluates a spectrum of ML algorithms — from classical linear methods to state-of-the-art gradient boosting frameworks. The system embodies core AI concepts including:

- Supervised learning with labeled tabular data
- Feature preprocessing and normalization
- Model training, validation, and generalization
- Performance evaluation using quantitative metrics
- Ensemble learning and boosting strategies
- Human-AI interaction through an intelligent GUI

1.3 Objectives

The primary objectives of this project are:

1. Develop a cross-platform desktop GUI for no-code ML model comparison on custom CSV datasets.
2. Integrate eleven diverse regression algorithms into a unified pipeline with consistent preprocessing.
3. Provide interactive visualizations including prediction plots, residual charts, ratio analysis, and 3D comparisons.
4. Implement a train/test ratio analysis framework to study model stability across different data splits.
5. Enable export of results, metrics, and charts to support further analysis.
6. Design an accessible quick-start guide for beginners with no programming background.

1.4 Scope of the Project

The project is scoped to regression tasks on numerical tabular data provided in CSV format. It covers:

- Any CSV file with a numerical target column (any column as per your preference)

- Eleven regression algorithms from scikit-learn, XGBoost, and LightGBM
- Train/test split ratios ranging from 60/40 to 80/20 and custom configurations
- Visualization of up to 11 models simultaneously in comparative charts

The system is currently scoped to regression (continuous output) and does not support classification, deep learning models, or real-time data streams. It runs locally on any machine with Python 3.8+ and the required dependencies.

2. Methodology

2.1 AI Techniques Used

The project employs classical Machine Learning (ML) techniques for supervised regression. No deep learning is used, as the focus is on interpretable, fast-training algorithms suited to tabular data. The primary techniques include:

- Rule-based evaluation: R^2 score thresholds (EXCELLENT ≥ 0.99 , VERY GOOD ≥ 0.90 , GOOD ≥ 0.75 , FAIR ≥ 0.50) to classify model quality
- Ensemble Learning: Random Forest, Extra Trees, Bagging (Bootstrap Aggregation)
- Gradient Boosting: Gradient Boosting Regressor, AdaBoost, XGBoost, LightGBM
- Instance-based Learning: K-Nearest Neighbours Regression
- Kernel Methods: Support Vector Regression (RBF kernel)
- Linear Models: Ordinary Least Squares Linear Regression
- Tree Models: Decision Tree Regression (CART algorithm)

2.2 Model Selection

Eleven models were selected to provide a comprehensive spectrum of algorithmic approaches:

Model	Type	Key Hyperparameters
Linear Regression	Linear	Default (no hyperparameters)
Decision Tree	Tree	random_state=42
Random Forest	Ensemble (Bagging)	n_estimators=250, random_state=42
Bagging	Ensemble (Bagging)	n_estimators=250, base=DecisionTree
Gradient Boosting	Ensemble (Boosting)	random_state=42
KNN	Instance-based	n_neighbors=5
SVR	Kernel Method	kernel=RBF, C=50, epsilon=0.02
Extra Trees	Ensemble (Random)	n_estimators=250, random_state=42
AdaBoost	Ensemble (Boosting)	random_state=42
LightGBM	Gradient Boosting	n_estimators=350, lr=0.05, leaves=31
XGBoost	Gradient Boosting	n_estimators=300, lr=0.05, depth=6

Table 1 — Supported Regression Models and Parameters

2.3 Algorithms and Mathematical Background

2.3.1 Linear Regression

Ordinary Least Squares minimizes the residual sum of squares:

$$\hat{y} = X\beta, \text{ where } \beta = (X^T X)^{-1} X^T y$$

2.3.2 Decision Tree Regression

CART algorithm recursively splits feature space by minimizing Mean Squared Error at each node:

$$\text{MSE}(\text{node}) = (1/n) \sum (y_i - \bar{y})^2$$

2.3.3 Random Forest / Extra Trees / Bagging

Bootstrap aggregation trains T trees on random subsets and averages predictions:

$$\hat{y}_{\text{RF}} = (1/T) \sum_i f_i(x), \text{ where } f_i \text{ trained on bootstrapped sample } B_i$$

Extra Trees further randomizes split thresholds for each feature, reducing variance at the cost of slightly higher bias.

2.3.4 Gradient Boosting (GBM, XGBoost, LightGBM, AdaBoost)

Sequentially fits residuals of the ensemble so far:

$$\begin{aligned} F_0(x) &= \bar{y} \\ F_m(x) &= F_{m-1}(x) + \eta \cdot h_m(x) \\ \text{where } h_m &= \arg\min \sum L(y_i, F_{m-1}(x_i) + h(x_i)) \end{aligned}$$

XGBoost and LightGBM add second-order Taylor expansion of the loss, L1/L2 regularization, and histogram-based split finding for efficiency. AdaBoost reweights samples based on previous residuals.

2.3.5 Support Vector Regression (SVR)

SVR fits a tube of width ε around the data, minimizing the ε -insensitive loss:

$$\begin{aligned} \min \quad & (1/2) \|w\|^2 + C \cdot \sum (\xi_i + \xi_i^*) \\ \text{s.t.} \quad & y_i - f(x_i) \leq \varepsilon + \xi_i, \quad f(x_i) - y_i \leq \varepsilon + \xi_i^* \\ \text{RBF Kernel:} \quad & K(x, x') = \exp(-\gamma \|x - x'\|^2) \end{aligned}$$

2.3.6 K-Nearest Neighbours Regression

$$\hat{y}(x) = (1/k) \sum_{i \in N_k(x)} y_i, \text{ where } N_k \text{ is the } k=5 \text{ nearest neighbours by Euclidean distance}$$

2.4 Dataset Details

The system accepts any CSV file with numerical features and a numerical target column (default: last column). The dataset handling pipeline includes:

- Source: User-supplied CSV file loaded via file dialog (default: data.csv)
- Size: No hardcoded limit — tested on datasets from 50 to 10,000+ rows
- Pre-processing: StandardScaler (zero mean, unit variance) applied within scikit-learn Pipeline to prevent data leakage

- Train/Test split: Default 80/20, configurable; ratio analysis supports 60-40, 70-30, 80-20 and custom splits
- Target column: Configurable via dropdown; defaults to last column
- Handling: Missing values and non-numerical columns are flagged; the application displays the top 10 best-performing feature combinations from actual data

2.5 Tools and Libraries Used

Tool / Library	Version	Purpose
Python	3.8+	Core language
Tkinter	Built-in	Desktop GUI framework
scikit-learn	≥ 1.0	ML algorithms, Pipeline, metrics
NumPy	≥ 1.21	Numerical computations
Pandas	≥ 1.3	Data loading & manipulation
Matplotlib	≥ 3.4	All charts and visualizations
XGBoost	≥ 1.6	XGBoost regressor (optional)
LightGBM	≥ 3.0	LightGBM regressor (optional)

Table 5 — Tools and Libraries Summary

Note: XGBoost and LightGBM fall back to HistGradientBoostingRegressor if not installed, ensuring the application runs in any Python environment.

3. System Design and Architecture

3.1 Block Diagram / Flowchart

The system follows a three-layer architecture: Presentation Layer (GUI), Business Logic Layer (ML Pipeline), and Data Layer (CSV Dataset). The high-level block diagram is described below:

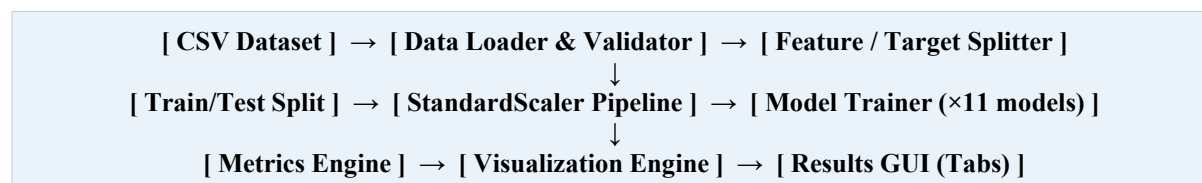


Figure 1 — System Architecture Block Diagram

3.2 Data Pipeline

The data pipeline covers the full journey from raw CSV to model output:

7. Collection: User selects a CSV file via file dialog. Data is read into a Pandas DataFrame.
8. Validation: Column headers are verified; target column is identified and confirmed as numerical.
9. Processing: Features (X) and target (y) are separated. Train/test split is applied using `sklearn.model_selection.train_test_split` with the configured ratio and `random_state=42`.
10. Scaling: `StandardScaler` is fitted on training data only and applied to both train and test sets within the scikit-learn Pipeline, preventing data leakage.
11. Model Training: Each selected model is wrapped in a Pipeline and fitted on scaled training data.
12. Prediction: Models predict on the test set. Residuals (actual - predicted) are computed.
13. Output: Metrics (R^2 , RMSE, MSE), prediction DataFrames, and ratio analysis results are stored and rendered in the GUI tabs.

3.3 Modules and Components

- `ModelWorkbenchGUI` class: Main application controller — manages GUI state, widget construction, and event handling
- Model Builders (`MODEL_BUILDERS` dict): Factory functions that instantiate each algorithm with its configured hyperparameters; gracefully falls back for optional libraries
- Training Engine (`_train_single_model`, `_run_ratio_analysis`): Executes model fitting and metric computation
- Visualization Engine (Matplotlib-based methods): Renders bar charts, scatter plots, residual plots, ratio analysis, 3D comparisons, and dataset top-10 charts
- Canvas Interaction Manager: Handles hover tooltips, rectangle-select zoom, and right-click zoom reset via Matplotlib event connections
- Export Manager: Exports metrics tables and charts to CSV and image files

- Quick Start Guide: Built-in text overlay explaining the workflow and metric interpretation for first-time users

4. Implementation

4.1 Code Snippets

4.1.1 Model Builder with Graceful Fallback (XGBoost)

The following snippet demonstrates how XGBoost is imported at runtime and falls back to HistGradientBoostingRegressor if unavailable, ensuring portability:

```
def build_xgboost_regressor():
    global XGBOOST_FALLBACK_MESSAGE
    try:
        xgb_module = importlib.import_module('xgboost')
        xgb_regressor_class = getattr(xgb_module, 'XGBRegressor')
        XGBOOST_FALLBACK_MESSAGE = None
    except (ImportError, AttributeError):
        XGBOOST_FALLBACK_MESSAGE = ('xgboost not available; using HistGradientBoosting.')
        return HistGradientBoostingRegressor(
            learning_rate=0.05, max_iter=350, max_depth=6,
            min_samples_leaf=20, random_state=RANDOM_STATE)
    return xgb_regressor_class(
        n_estimators=300, learning_rate=0.05, max_depth=6,
        subsample=0.9, colsample_bytree=0.9,
        objective='reg:squarederror', random_state=RANDOM_STATE, n_jobs=-1)
```

4.1.2 Training Pipeline with Metrics

Each model is wrapped in a StandardScaler + Model pipeline and evaluated with three regression metrics:

```
def _build_model_pipeline(self, model_name: str):
    model = MODEL_BUILDERS[model_name]()
    return Pipeline([('scaler', StandardScaler()), ('model', model)])

def _train_single_model(self, model_name, X_train, X_test, y_train, y_test):
    pipeline = self._build_model_pipeline(model_name)
    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)
    metrics = self._compute_regression_metrics(y_test, y_pred)
    pred_df = pd.DataFrame({
        'Actual': y_test.reset_index(drop=True),
        'Predicted': pd.Series(y_pred),
        'Residual': y_test.reset_index(drop=True) - pd.Series(y_pred),
    })
    return pipeline, metrics, pred_df

def _compute_regression_metrics(self, y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    return {'MSE': float(mse), 'RMSE': float(np.sqrt(mse)),
            'R2': float(r2_score(y_true, y_pred))}
```

4.1.3 Ratio Analysis

The ratio analysis iterates over multiple train/test splits to study model stability:

```
def _run_ratio_analysis(self, X, y, model_name, train_sizes):
    rows = []
    for train_size in train_sizes:
        X_tr, X_te, y_tr, y_te = train_test_split(
```

```

X, y, train_size=train_size, random_state=RANDOM_STATE)
_, metrics, _ = self._train_single_model(
    model_name, X_tr, X_te, y_tr, y_te)
rows.append({'Model': model_name,
             'Ratio': self._ratio_label(train_size),
             **metrics})
return pd.DataFrame(rows)

```

4.2 Model Training Process

The training process follows these steps:

14. The user loads a CSV file and optionally selects which models to train via checkboxes.
15. On clicking 'TRAIN MODELS', the GUI spawns a background thread to prevent UI freezing.
16. Each selected model's pipeline is built, fitted, and evaluated. Progress is displayed in the status bar.
17. After training, results are stored in self.metrics_df and rendered across multiple tabs: Summary, Metrics Table, Charts, Predictions, Ratio Analysis, and Dataset Top 10.
18. For ratio analysis, models are re-trained on each configured split ratio (default: 80/20, 70/30, 60/40) and results are compared.

4.3 Evaluation Metrics

Three quantitative regression metrics are computed for every model:

Metric	Formula	Interpretation
R ² Score	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	Proportion of variance explained; 1.0 = perfect
RMSE	$\sqrt{(\sum (y_i - \hat{y}_i)^2)/n}$	Average prediction error in target units; lower = better
MSE	$\sum (y_i - \hat{y}_i)^2/n$	Mean squared error; penalizes large errors heavily

Table 6 — Evaluation Criteria

R² quality thresholds applied by the system:

- $R^2 \geq 0.99 \rightarrow \text{EXCELLENT}$ (near-perfect prediction)
- $0.90 \leq R^2 < 0.99 \rightarrow \text{VERY GOOD}$ (excellent accuracy)
- $0.75 \leq R^2 < 0.90 \rightarrow \text{GOOD}$ (very useful)
- $0.50 \leq R^2 < 0.75 \rightarrow \text{FAIR}$ (acceptable)
- $R^2 < 0.50 \rightarrow \text{POOR}$ (not very useful)

5. Discussion

5.1 Summary of Work

This project successfully delivered a fully functional ML Model Workbench as a desktop Python application. The system integrates eleven regression algorithms in a unified framework with automatic preprocessing, interactive visualization, and export capabilities. Key accomplishments include:

- A responsive Tkinter GUI with six tabbed views (Summary, Metrics Table, Charts, Predictions, Ratio Analysis, Dataset Top 10)
- Standardized Pipeline architecture ensuring no data leakage across all models
- Interactive Matplotlib charts with hover tooltips, rectangle-select zoom, and right-click reset
- Full 2D and 3D model comparison visualizations
- Configurable train/test ratio analysis across up to 6 split configurations
- Graceful degradation for optional libraries (XGBoost, LightGBM) with automatic fallback
- Built-in Quick Start Guide and metric interpretation for new users

5.2 Challenges Faced

- Thread Safety: Tkinter is not thread-safe; training in a background thread required careful use of thread-safe GUI update patterns (after() callbacks) to prevent race conditions.
- Data Leakage Prevention: Ensuring StandardScaler was fitted only on training data required wrapping it inside scikit-learn Pipelines rather than applying it globally.
- Optional Dependencies: XGBoost and LightGBM are not universally available. Implementing runtime import with HistGradientBoostingRegressor fallback required careful module introspection via importlib.
- Canvas Hover Performance: Attaching per-artist hover text and managing Matplotlib annotation visibility across multiple dynamic canvases required a custom state-tracking system (_canvas_interactions dictionary).
- Multi-ratio Analysis Scaling: Training 11 models across 6+ ratios (66+ training runs) had to be managed without blocking the UI while providing real-time progress feedback.

5.3 Success Criteria Met

- All 11 regression models train successfully on any well-formed CSV dataset ✓
- R^2 , RMSE, and MSE metrics computed and displayed for every model ✓
- Interactive charts render correctly with hover tooltips and zoom ✓
- Ratio analysis completes across configured train/test splits ✓
- Application runs without XGBoost or LightGBM installed ✓
- Quick Start Guide accessible from the GUI ✓

5.4 Limitations

- Regression only: The system does not support classification, clustering, or deep learning tasks.
- Local execution: No cloud or server deployment; performance is bounded by the user's hardware.
- No hyperparameter tuning UI: Hyperparameters are fixed in code; users cannot tune them from the GUI without modifying `model_gui.py`.
- No missing value imputation: CSV files with missing values must be pre-cleaned before loading.
- Memory constraints: Very large datasets (millions of rows) may cause memory issues on low-RAM machines.
- Single target column: Multi-output regression is not supported.

6. Future Work

6.1 Scope for Improvements

- Hyperparameter Tuning UI: Add sliders and dropdowns for each model's key hyperparameters, with live re-training on change.
- Classification Support: Extend to classification tasks with models like Logistic Regression, SVM, and neural networks; add accuracy, F1, ROC-AUC metrics.
- Deep Learning Integration: Incorporate TensorFlow/Keras MLP regressor as an optional model.
- AutoML Mode: Automatically run all models and rank them; suggest the best model for the dataset characteristics.
- Missing Value Handling: Add imputation strategies (mean, median, KNN imputer) within the data pipeline.
- SHAP Explainability: Integrate SHAP (SHapley Additive exPlanations) for feature importance visualization per model.
- Cross-Validation: Replace single train/test split with k-fold cross-validation for more robust evaluation.
- Report Export: Auto-generate a PDF report summarizing dataset stats, model performance, and recommended model.

6.2 Possible Real-World Deployment Strategies

- Web Application: Port the GUI to a browser-based interface using Streamlit or Dash (Plotly), enabling team collaboration without installation.
- Docker Container: Package the application and all dependencies in a Docker image for consistent deployment across OS platforms.
- Cloud SaaS: Deploy as a multi-tenant web service on AWS/GCP with user authentication, stored datasets, and job queuing for large-scale training.
- REST API: Expose model training and prediction as REST endpoints, enabling integration with third-party analytics tools and dashboards.
- Edge Deployment: Export the best-trained model as a ONNX or Pickle file for deployment on IoT devices or embedded systems.
- Academic LMS Integration: Package as a Jupyter widget or Colab notebook extension for use in university courses on Machine Learning.

Appendix

A. Full Code

The complete source code of `model_gui.py` (2,014 lines) is available in the project repository. Key architectural sections are:

Lines	Section	Description
1–30	Imports & Constants	Library imports, global constants, default values
31–80	Quick Start Guide	Embedded help text string for first-time users
81–200	Model Builders	Factory functions for all 11 models with fallback logic
201–250	ModelWorkbenchGUI.__init__	GUI state initialization and window setup
251–600	Canvas Interactions	Hover, zoom, and annotation management system
601–900	GUI Layout (_build_*)	All tab and widget construction methods
901–1200	Training Engine	Model training, pipeline construction, ratio analysis
1201–1800	Visualization Engine	All chart rendering methods for each tab
1801–2014	Entry Point	main() function, Tk root window creation

Table A.1 — Source Code Structure

B. Project Links

Deployed project link:-

<https://ml-model-analyzer-1.onrender.com/>

Youtube Link:

<https://youtube.com/shorts/ozGPHmMgxiw?feature=share>

Git-Hub Link:

<https://github.com/bhavin1369/ml-model-analyzer.git>

C. Additional Information

C.1 Installation Requirements

```
# Python 3.8+ required
pip install numpy pandas matplotlib scikit-learn
pip install xgboost lightgbm # optional — fallback available

# Run the application
python model_gui.py
```

C.2 Supported Operating Systems

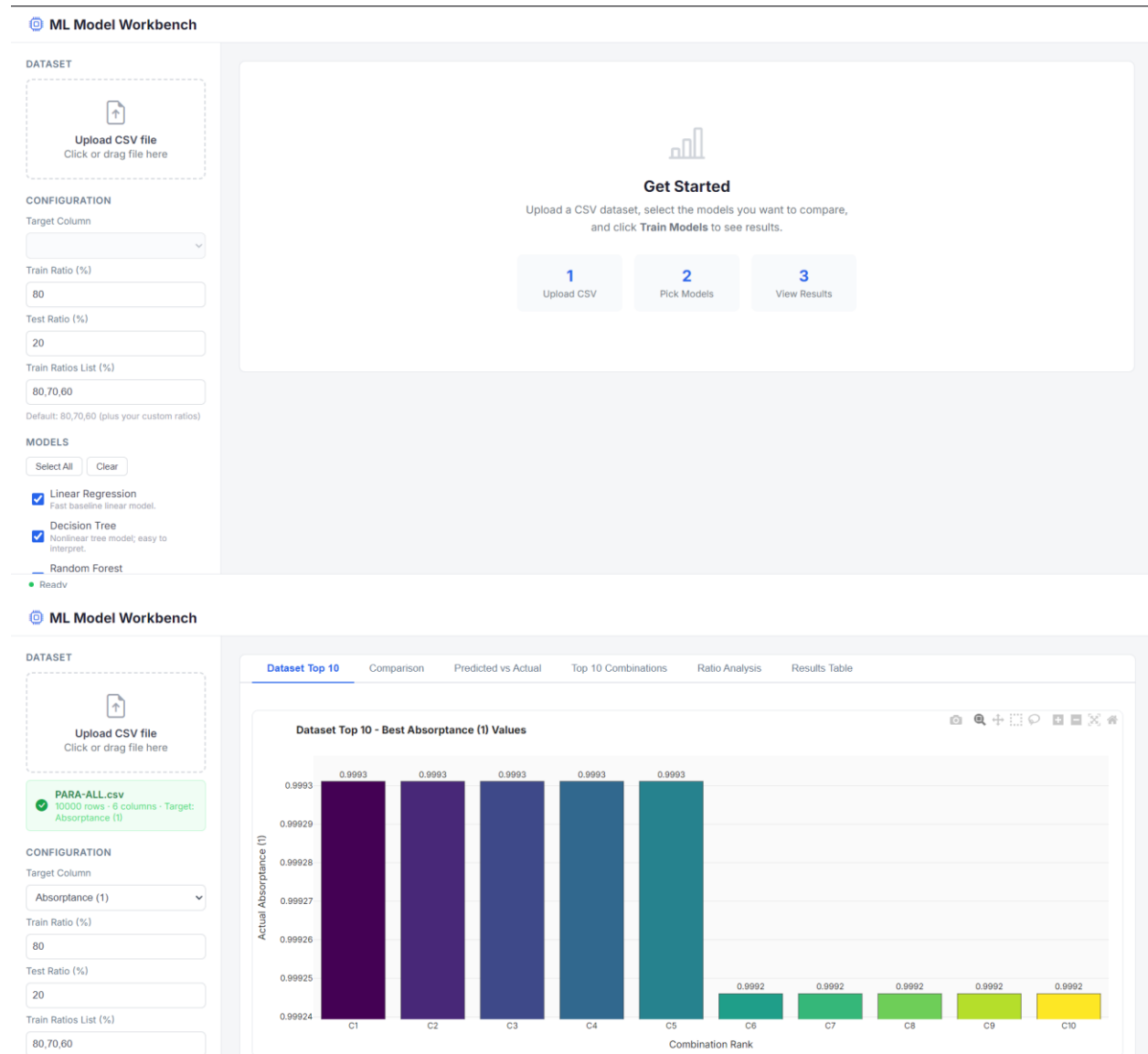
- Windows 10/11 (primary development platform)

- macOS 11+ (Big Sur and later)
- Linux (Ubuntu 20.04+, any distribution with Python Tkinter support)

C.3 Known Fallback Behaviour

If XGBoost is not installed, the XGBoost model automatically uses HistGradientBoostingRegressor (scikit-learn) with equivalent hyperparameters. The same fallback applies to LightGBM. A warning message is displayed in the GUI status bar when a fallback is active.

Ui Screen Shots:



MODELS

Select AllClear

Linear Regression

Fast baseline linear model.



Decision Tree

Nonlinear tree model; easy to interpret.



Random Forest

Bagged trees; stable and strong for tabular data.



Bagging

Bootstrap aggregation with decision-tree base learners.



Gradient Boosting

Boosted trees; often strong accuracy.



KNN

Distance-based model; sensitive to scaling.



SVR

Support Vector Regression with RBF kernel.



Extra Trees

Randomized tree ensemble; robust and fast.



AdaBoost

Boosting model using weighted weak learners.



XGBoost

Boosted tree ensemble with strong performance on tabular datasets.



LightGBM

Histogram-based gradient boosting for fast, accurate tabular modeling.

▶ **Train Models**

● Dataset loaded: PARA-ALL.csv

